CS141 CW2 Report

1 Parts Zero to Three

To implement the memory functionality, the main REPL loop is a `StateT` monad transformer type containing a `Memory` type as the state and an `IO ()` monad type as the result value. This allows the memory state to be binded through repeated calls of the REPL and so the memory can be updated sequentially as items are remembered. The memory is implemented as a `Data.Map` (good for storing key value pairs) where the key is either a string or a special `ThatKey` constructor which allows for differentiation of the special "that" keyword used in expression evaluation (allows for remembering of a key value pair where the key is "that" which does not interfere with the last evaluated expression "that"). In terms of separation of concerns, the program is split into two parts: parsing the request given and responding to the request. The parsing of the request purely parses the request and hence is not passed the memory map. The response involves other computation such as evaluating expressions and retrieving values from the memory. For the expression evaluation, the parser first attempts to parse an expression with three terms (two operators) with first two being values and the third being an expression. If this fails, it attempts to parse an expression with two terms (one operator) where both are values. If this fails, it parses a single value. This emulates the regex `expr = (x([+-*]y)?)([+-*]expr)?` and produces a left associative expression tree as required.

2 Beyond the Specification

With CS141's monadic parsing and CS143's formal grammars, I decided that the natural extension to this would be to produce a programming language and a compiler (which can be called as a command in λGPT) for this programming language in Haskell using monadic parsing. The programming language is called Little Language and is a statically typed procedural language that is similar to C. It only contains a very limited set of language constructs and is missing certain features such as: scope, arrays and functions. However, it does contain conditional statements, integers, addition, equality comparisons and booleans. I think with the addition of arrays, it would be capable of simulating a Turing machine and would be Turing complete. The code for the compiler is under `src/Compiler`. The compiler uses Megaparsec to parse the source code file into a token stream. It then uses Megaparsec again to parse the token stream into a parse tree according to the context-free grammar of Little Language (see Figure 1). It also produces a symbol table (passed through functions by a `StateT` monad transformer wrapping the `Parsec` monad) in order to type check variables. From this, it maps each section of the parse tree to x86-64 assembly code to produce an assembly translation of the file. It then calls the assembler (NASM which is installed on DCS machines) and then the linker to produce an executable file. It is integrated into λGPT through the `Compile [path to source file] [path to executable destination]` command e.g. `Compile ./sample_programs/fib.ll ./fib`. This prints the parse tree and produces the executable. For demonstration purposes, my submission contains a folder called `sample_programs` which contains some `.ll` files which are Little Language source code files. The Fibonacci one (`./sample_programs/fib.ll`) is likely of most interest. One of the shortcomings of my compiler is that it is not yet capable of outputting meaningful error messages. If it fails to compile, it will not say why or where it failed. With more time, I would like to improve upon this. My x86-64 assembly code is also not the nicest (but hopefully this is insignificant for a Haskell coursework). The only additional library required by my compiler is `process` in order to execute commands and this has been added to `package.yaml`. In terms of personal experience, I feel that Haskell

```
Prog    ::=  <Block>
Block   ::= <Stmt> <Block>
         | epsilon
Stmt    ::= <Decl>
         | if(<Expr>){<Block>}
         | <Asg>
         | <Out>
         | while(<Expr>){<Block>}
Decl    ::= <Type> var;
Asg     ::= var SingleEquals <Expr>;
Out     ::= printl <Expr>;
Expr    ::= var
         | numlit
         | <Expr> <BinOp> <Expr>
BinOp   ::= Plus
         | DoubleEquals
         | NotEqual
Type    ::= Int
         | Bool
```

Figure 1: Syntax of Little Language in BNF

was very suited to this application and I enjoyed using monads and other features of Haskell in interesting ways to write concise and clean code. I have not made a compiler before and I enjoyed the experience. I did not enjoy writing assembly.

3 Resources used

For parts zero to three, I relied on Hoogle and Hackage.
For the compiler, I used:

- "Engineering a Compiler" by Keith Cooper for an introduction into the architecture of compilers and help with constructing the syntax for the language.
- Creating a Compiler YouTube video series by Pixeled for inspiration on how to construct the code generation part of the compiler.
- Getting Started with x86-64 Assembly on Linux by Gabriele N. Tornetta for help with x86-64.